Министерство науки и высшего образования Российской Федерации

ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ

НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО

ITMO University

ПРАКТИЧЕСКАЯ РАБОТА №2

По дисциплине Контейнеризация и оркестрация

Тема работы Знакомство с экосистемой Docker: от базовых команд к диагностике сервиса

Обучающийся Ямалиева Альбина Азатовна

Факультет прикладной информатики

Группа K3320

Направление подготовки 11.03.02 Инфокоммуникационные технологии и системы связи

Образовательная программа Программирование в инфокоммуникационных системах

Обучающийся	_____	_____	Ямалиева А.А.
	(дата)	(подпись)	(Ф.И.О.)
Руководитель	_____	_____	Аминов Н.С.
	(дата)	(подпись)	(Ф.И.О.)

СОДЕРЖАНИЕ

Введение 3

1. Первый рабочий образ 4

2. Оптимизация образа 6

3. .dockerignore 7

4. Healthcheck 9

5. Финальный артефакт и тегирование 10

Заключение 11

Введение

Цель работы – освоить создание собственных Docker-образов с помощью Dockerfile, изучить передачу переменных окружения, научиться оптимизировать образ, использовать .dockerignore, настраивать HEALTHCHECK, а также выполнять тегирование итоговых образов.

1. Первый рабочий образ

Номер ИСУ	472706
Персональный порт	5430 + последние 2 цифры ИСУ: 5430 + 06 = 5436
Персональный префикс	Фамилия латиницей в нижнем регистре: yamalieva
Секретный ключ	secret06yamalieva

В Dockerfile(Рисунок 1) описан базовый образ приложения. В качестве основы использовался образ python:3.11-slim, содержащий Python 3.11 в облегчённом Linux-окружении. Затем с помощью LABEL были добавлены метаданные образа. Командой WORKDIR /app была задана рабочая директория внутри контейнера. После этого в образ копировался файл requirements.txt, содержащий зависимости Python-приложения, и выполнялась их установка командой RUN pip install -r requirements.txt. Далее в образ копировалась папка app с исходным кодом приложения. Команда EXPOSE указывала порт, на котором приложение работает внутри контейнера. В CMD задавалась команда запуска сервера uvicorn, который запускал объект app из модуля app.main, принимал подключения на всех интерфейсах 0.0.0.0 и использовал персональный порт. По методичке именно такие элементы требовались в базовом Dockerfile.

```dockerfile
Dockerfile > ...
1   FROM python:3.11-slim
2   #Метаданные
3   LABEL maintainer="yamalieva@lab.local"
4   LABEL version="1.0"
5
6   WORKDIR /app
7
8   #файл зависимостей
9   COPY requirements.txt .
10
11  #зависимости Python
12  RUN pip install -r requirements.txt
13
14  COPY app/ ./app/
15
16  EXPOSE 5436
17
18  #Запуск
19  CMD ["uvicorn", "app.main:app", "--host", "0.0.0.0", "--port", "5436"]
```

Рисунок 1 – Dockerfile

Для проверки работоспособности приложение используем curl(Рисунки 2-3).



Рисунок 2 – Проверка образа



Рисунок 3 – Проверка образа

Также перешла по ссылки с моим хостом, чтобы проверить в браузере(Рисунок 4).

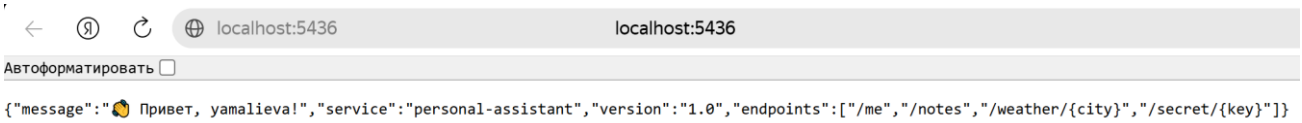


Рисунок 4 – Приложение в браузере

2. Оптимизация образа

В файле Dockerfile.optimized((Рисунок 7) была выполнена оптимизация установки зависимостей. Файл requirements.txt копировался отдельно от исходного кода приложения, что позволяет Docker повторно использовать закэшированный слой с установленными библиотеками, если список зависимостей не изменился. Для установки пакетов использовалась команда pip install --no-cache-dir -r requirements.txt, где параметр --no-cache-dir предотвращает сохранение кэша pip внутри образа. Это позволяет уменьшить количество лишних данных в образе и ускорить повторные сборки при изменении только исходного кода приложения.

По новому файлу собираем образ и видим что после оптимизации размер образа уменьшился на 12,4 МБ. Уменьшение связано с использованием pip install --no-cache-dir, который не сохраняет кэш установленных пакетов внутри образа(Рисунок 6)

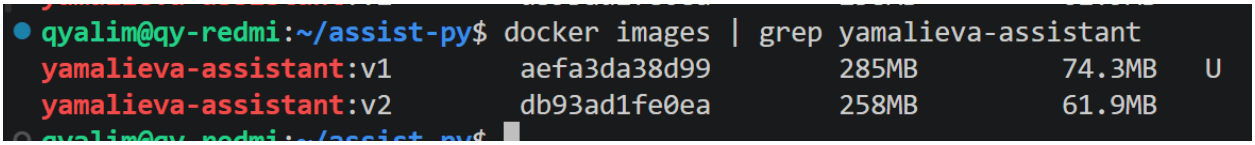


Рисунок 5 – Сравнение двух образов

Отдельное копирование requirements.txt ускоряет повторные сборки, потому что Docker кэширует слои. Если зависимости не изменились, слой с pip install берётся из кэша, а пересобирается только слой с изменённым кодом приложения. Поэтому изменение бизнес-логики не приводит к повторной установке всех зависимостей

3. .dockerignore

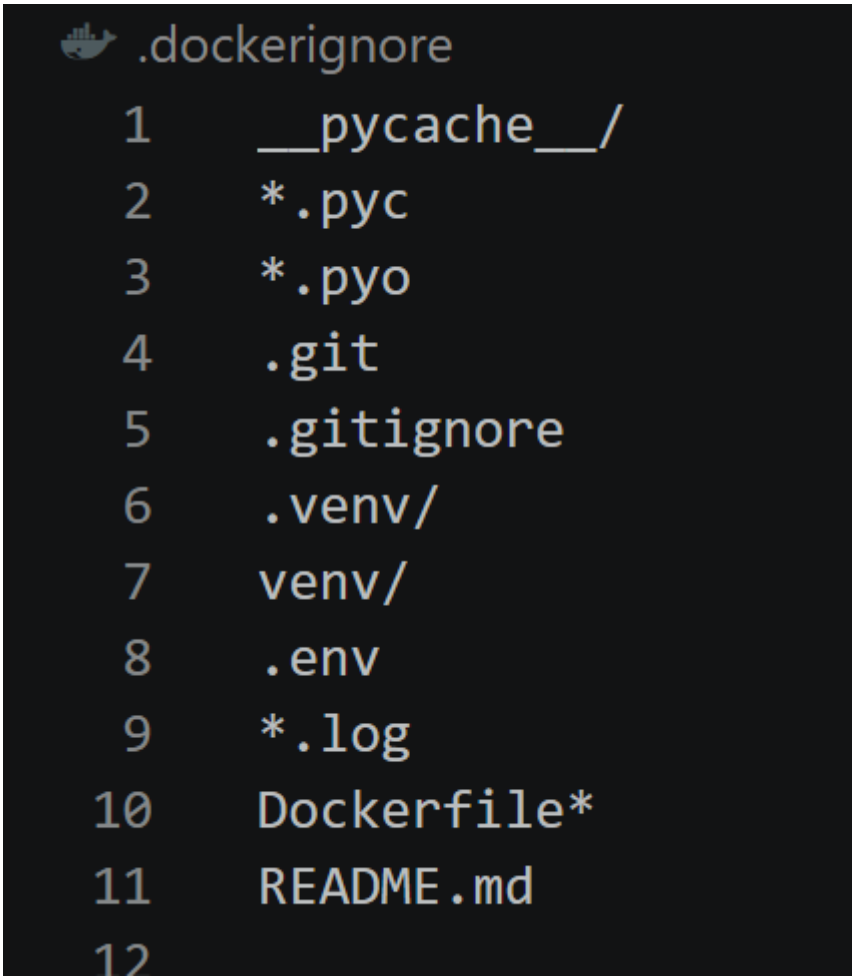


Рисунок 6 – .dockerignore

Без файла .dockerignore в контекст сборки и потенциально в образ могли попасть служебные и лишние файлы проекта: каталог .git, .gitignore, виртуальное окружение .venv/ или venv/, файлы .env, логи *.log, кэш Python __pycache__/, скомпилированные файлы *.pyc и *.pyo, а также README.md и Dockerfile-файлы. В методичке именно эти элементы предлагается исключить через .dockerignore(Рисунок 2).

Наличие .git в образе является потенциальной уязвимостью, потому что каталог .git содержит историю репозитория и служебную информацию Git. В истории могут сохраниться ранее удалённые файлы, старые версии конфигураций, адрес удалённого репозитория и, если когда-то по ошибке были закоммичены секреты, они тоже могут остаться в истории. Поэтому .git не

нужен работающему приложению и его лучше не включать в Docker-контекст и образ.

4. Healthcheck

В финальной версии Dockerfile.optimized была добавлена проверка состояния приложения с помощью HEALTHCHECK. Docker периодически обращается к эндпоинту /health и по результату определяет состояние контейнера как healthy или unhealthy. Для выполнения проверки в образ также был установлен curl. Остальная структура оптимизированного Dockerfile – отдельное копирование requirements.txt, установка зависимостей с --no-cache-dir, копирование приложения и запуск через uvicorn – сохранилась без изменений(Рисунок 7).

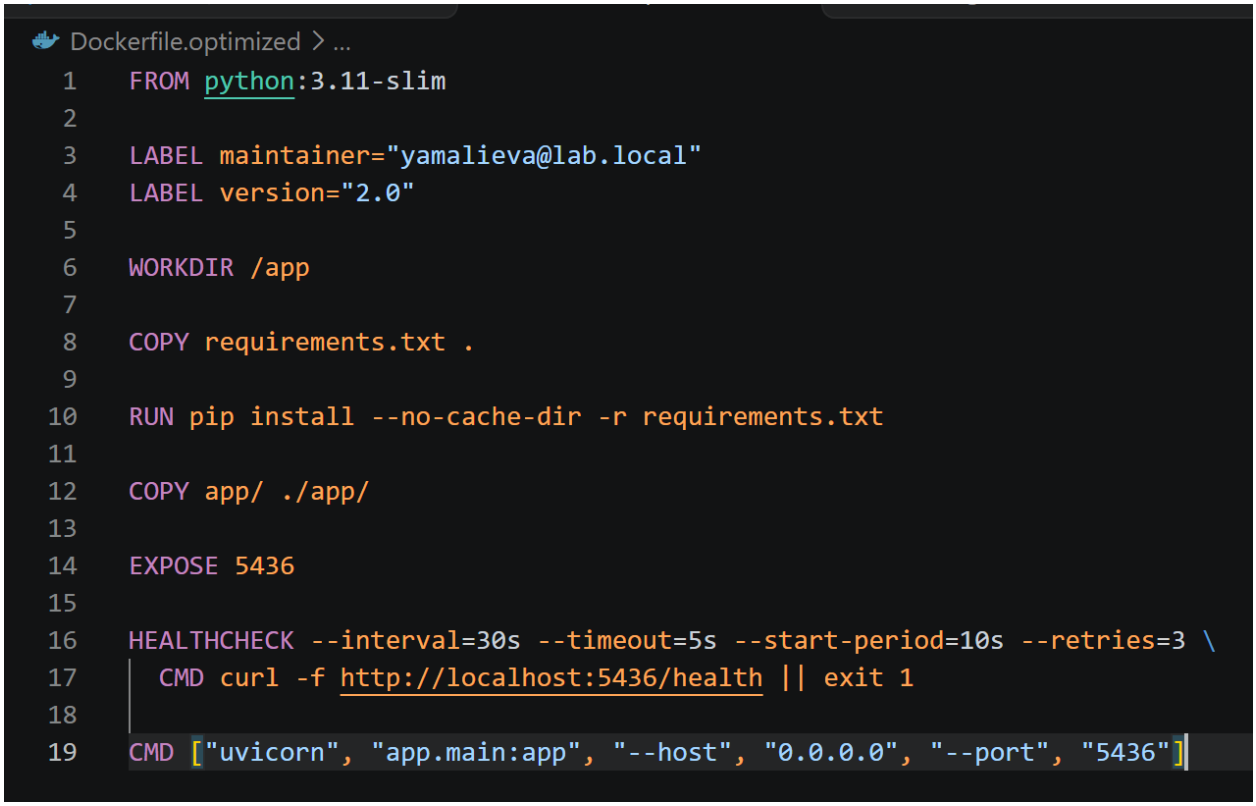


Рисунок 7 – Dockerfile.optimized

Проверяем статус «здоровья»(Рисунок 8).



Рисунок 8 – Финальный контейнер

5. Финальный артефакт и тегирование

Выводим финальный артефакт с логином и хостом, временем и нашими образами(Рисунок 9).

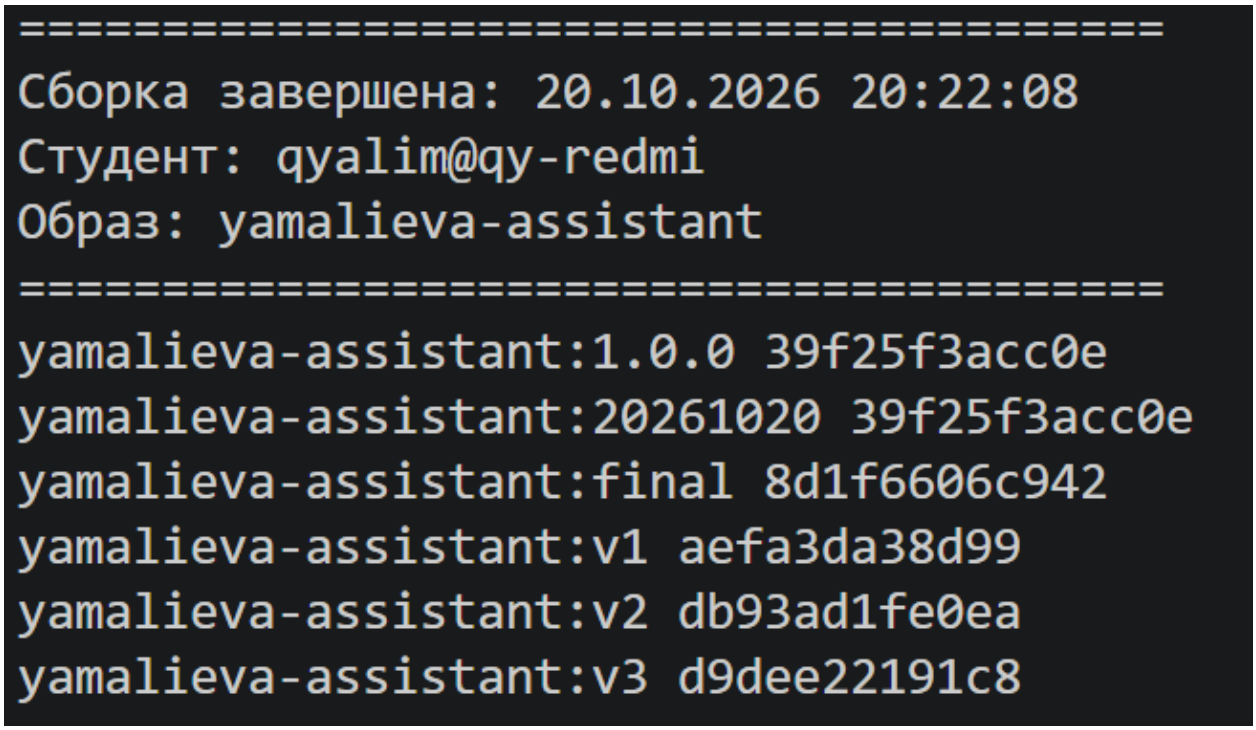


Рисунок 9 – Финальный артефакт

Заключение

В ходе практической работы был создан Docker-образ для Python-приложения, выполнена его сборка и запуск контейнера с передачей персональных переменных окружения и пробросом порта. Была проверена работоспособность основных эндпоинтов приложения. Затем образ был оптимизирован за счёт более эффективной установки зависимостей, создан файл .dockerignore для исключения лишних файлов из контекста сборки, а также добавлена проверка состояния сервиса с помощью HEALTHCHECK. На заключительном этапе финальному образу были назначены дополнительные теги версии и даты сборки. В результате были освоены базовые приёмы создания, оптимизации и подготовки Docker-образов к дальнейшему использованию и оркестрации.